

1. AI 協作流程與策略

1.1 專案背景與任務說明

四項新功能需求明細如下：

#	功能	說明
1	Dependency Line (虛線箭頭)	新增虛線箭頭連線類型，以 BasicStroke 虛線繪製。
2	Port 紅點提示	在任何建立 Line 的模式下，滑鼠移到物件附近 (Port 範圍內) 時顯示紅色圓點提示，離開則消失。
3	拖曳建立物件	在 Class 或 UseCase 模式下，拖曳起迄點不同時可決定物件大小 (需限制最小間距，單純點擊則維持預設大小)。
4	Port 點擊高亮連線	在 Select 模式下，點擊 Port 將與其相連的線條高亮為藍色，點擊空白處或未連線 Port 需取消高亮。

1.2 協作流程概覽與對話歷程

在開發歷程中，我們採取引導與漸進式實作策略。首先讓 AI 深入解析既有專案程式碼架構，而非直接盲目動手撰寫程式。接著要求 AI 自動生成架構分析與具體的 Tasks 清單。最後，在面對 Swing 繁複的 UI 事件傳遞 Bug 時，透過執行、截圖、提供錯誤細節描述、AI 診斷修正的 4 輪除錯循環，成功攻克所有核心難關。

【User Prompt 1 — 需求閱讀與記憶階段】

請你詳細閱讀此份文件，內容為本次要完成的新功能規格，請你先閱讀完，了解了所有的需求和規範之後再開始規劃。目前先幫我閱讀並記憶你所該遵守及可能規劃如何進行。

【AI 回覆與分析策略】

AI 閱讀規格文件後，精確拆解出四個功能需求，並特別記錄了禁止重構架構的硬性限制條件。確認了 FuncIndex 對應關係 (0=Select, 1=Association ... 6=DependencyLine) 以及 AreaDefine 常數 (TOP=3, RIGHT=2, LEFT=1, BOTTOM=0) 等關鍵底層系統細節後，才請求進入下一階段規劃。

【User Prompt 2 — 規格與 Task 文件生成】

幫我建立 spec 相關檔案，產生 specify 等等 md 檔，後續再生成應該要有的 task，最後再進行實作。

【AI 回覆與分析策略】

AI 在專案目錄中建立了 SPEC.md 與 TASKS.md 兩份文件，將四個複雜的功能拆解為具體的可執行 task (含目標檔案、新增方法、修改邏輯)，並以此作為後續實作的實體錨點，確保每個步驟都有明確的程式邊界與驗收標準。

【User Prompt 3 — 逐步實作指令】

幫我按照 task 逐步實作。

【AI 實作內容摘要】

AI 依序完成以下核心實作：

- 新建 DependencyLine.java (以 BasicStroke 實作虛線及開口三角箭頭繪製) 。
- 修改 FuncPanelHandler 新增 makeDependencyIcon() 、 getIcon(6) 、 getFunc(6) 以動態繪製工具列按鈕。
- 修改 Core.java 新增 isDependencyLine() 並更新基礎連線邏輯判斷。
- 修改 CanvasPanelHandler 核心繪圖邏輯，添加紅點 Port 提示與 Port 高亮選取線段狀態機制。

【User Prompt 4 — 第一輪截圖除錯 (Icon 破圖與虛線問題) 】

(附帶執行畫面截圖) 目前好像沒有虛線箭頭，都是實線的，且圖片會破圖，連線也很難連到，切換模式也常常失敗。幫我看一下要如何改，記得不要重構專案。

【AI 診斷與修正說明】

AI 發現虛線箭頭的工具列 icon 因使用 dependency_line.jpg 導致因缺乏外部圖檔而顯示錯誤。修正方案：直接改為在 FuncPanelHandler 中使用 Java 程式碼，以 BufferedImage + Graphics2D 搭配 BasicStroke 動態記憶體繪製虛線圖示，完全擺脫對外部靜態圖檔的依賴。

【User Prompt 5 — 第二輪截圖除錯 (拖曳功能與滑鼠移入提示) 】

(附帶執行畫面截圖) 目前沒辦法用拖曳大小的方式去建立 UseCase 跟 Class , 且連線部分常常失敗 , 滑鼠移過去也不會顯示紅點 , 不太好操作 , 幫我修改一下。

【AI 診斷與修正說明】

AI 診斷出兩個核心問題：

1. Swing 的 `MouseMotionListener` 預設不會從子元件向父級元件冒泡傳遞 , 因此滑鼠移到 Class 物件上時 , 底層 Canvas 無法感知。修正：在 `addObject` 時 , 動態為新建物件附加 `MouseMotionAdapter` 並透過 `SwingUtilities.convertPoint()` 轉換精確相對座標。
2. 拖曳物件釋放後 `mouseClicked` 仍會被觸發 , 導致在原位重複多建立一個預設大小的物件。修正：在 `CPHActionListener` 加入 `wasDragged` 狀態布林旗標。

【User Prompt 6 — 第三輪截圖除錯 (物件大小與紅點層級) 】

(附帶執行畫面截圖) 現在仍然無法用拖曳的方式建立物件 , 另外跟我講一下現在連接線段要怎麼連 , 另外還有一個問題是紅點目前好像不是在圖層最上面 , 會被遮住半個點 , 幫我改一下。

【AI 診斷與修正說明】

AI 發現：

1. `BasicClass` 與 `UseCase` 元件內部 `paintComponent()` 原始寫法死扣 `defSize` 變數 , 並未與當前 `width/height` 綁定。修正：改為使用 `this.getWidth()` / `this.getHeight()` 進行自適應縮放。
2. 紅點在 `paintComponent` 中繪製會因 Swing 的渲染順序被子元件遮擋。修正：覆寫 `paintChildren()` 方法 , 確保紅點在所有子元件繪製完畢後才覆蓋疊加 , 保證永遠位於最上層層級。

【User Prompt 7 — 第四輪大優化 (深層連線失效除錯) 】

(附帶執行畫面截圖) 好像還是連不太起來欸，我都等到有紅點出現才放開了。另外現在有一個 class 只能連一條等等的規定嗎？UseCase 可以連到其他的 UseCase 嗎？幫我統整目前的連線規則。

【AI 終極底層原因診斷與修正】

經 AI 深入分析 AWT LightweightDispatcher 機制：當用戶在物件內部按下鼠標時，事件被子元件攔截，底層 contextPanel 的 CPHActionListener 從未收到過 mousePressed 事件，導致 from 變數始終為 null，連線必定失敗！

修正策略：

1. 將 CPHActionListener 改為 CanvasPanelHandler 的成員欄位進行持久化控制。
2. 在 addObject() 中，為每個物件手動綁定專屬 MouseAdapter，將其按壓/釋放事件利用 SwingUtilities.convertPoint() 精準轉換並手動轉發 (forward) 給 cphListener。
3. 新增具有 ± 15 像素容許誤差的 isInsideWithTolerance() 演算，確保物件邊緣的模糊辨識判定。

2. 驗證歷程與程式修正明細

2.1 測試與驗證策略

每當 AI 修改程式碼後，立即在終端機執行 `javac` 進行編譯驗證，確保沒有破壞原有的封裝與類型系統。編譯成功後，直接運行 UML 系統，依照作業規範進行邊界條件測試（如：單擊物件、超小範圍拖曳、邊緣 Port 拖曳連線等），並利用 UI 截圖將不合規的表現直觀地回饋給 AI 進行分析。

2.2 關鍵 Bug 修正程式對照

【Bug 發現 — 拖曳建立物件後外觀大小未改變】

雖然在實作中已經正確呼叫並傳入自訂大小的 `setCustomSize()`，然而在 `BasicClass` 與 `UseCase` 的 `paintComponent()` 方法中，繪製圖形時依舊硬編碼使用了常數 `defSize`，造成畫面上形體完全不隨拖曳大小發生改變。

```
// 【修正前】 ( BasicClass.java )
g.fillRect(0, i * defSize.height, defSize.width - 1, defSize.height - 1);

// 【修正後】 ( BasicClass.java )
int w = this.getWidth();
int rowH = texts.size() > 0 ? this.getHeight() / texts.size() : defSize.height;
g.fillRect(0, i * rowH, w - 1, rowH - 1);
```

【Bug 發現 — Port 紅點提示被物件背景遮擋一半】

紅點原本是在 `contextPanel` 的 `paintComponent()` 方法結尾進行繪製。但在 Swing 的標準雙緩衝渲染架構中，`paintComponent` 會先於 `paintChildren` 執行，導致隨後被繪製出來的子元件（`Class/UseCase`）覆蓋遮擋住了紅點，造成嚴重的破圖和視覺遮蔽現象。

```
// 【修正前】
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    drawPortHints(g); // 在子元件之前繪製，會被遮擋
}

// 【修正後】
protected void paintChildren(Graphics g) {
    super.paintChildren(g);
    drawPortHints(g); // 改在子元件之後繪製，永遠置頂
}
```

【Bug 發現 — 元件內部無法啟動連線（Swing 事件攔截）】

當在 Class 內部點擊並嘗試拖曳連線時，AWT LightweightDispatcher 將點擊事件優先分派給了該 Class 元件。因為底層面板沒收到 mousePressed，連線起點始終為 null。這是導致作業功能連線極度不流暢、時常失敗的根本元凶。

```
// 【核心解法：事件轉發技術】
obj.addMouseListener(new MouseAdapter() {
    @Override
    public void mousePressed(MouseEvent e) {
        // 將子元件內部的滑鼠坐標，精準轉換為父級 contextPanel 的相對坐標
        Point p = SwingUtilities.convertPoint(obj, e.getPoint(), contextPanel);
        // 手動包裝並轉發事件給核心偵聽器，藉此啟用正確的連線判定機制
        cphListener.mousePressed(new MouseEvent(
            contextPanel, e.getID(), e.getWhen(),
            e.getModifiersEx(), p.x, p.y, e.getClickCount(), false));
    }
}
```

```
});
```

3. 心得與反思

我認為目前的軟體開發流程，藉由 AI 的協作已經可以獲得極大幅度的效率提升。過去如果要接手他人撰寫的專案，通常必須先花大量時間閱讀程式碼、理解專案架構、追蹤各個類別與方法之間的關係，才能開始進行修改。然而在 AI 的協助下，即使是一個自己並不熟悉的專案，也可以先請 AI 協助整理專案邏輯、分析程式架構、說明主要元件的運作流程，進而快速找出和需求相關的修改點與可能出問題的位置。這讓開發者在面對陌生程式碼時，不再需要完全從零開始摸索，而是能夠更有效率地建立對專案的整體理解。

此外，AI 不只可以協助寫程式，也能搭配其他軟體開發規劃工具，例如 SpecKit 這類規格導向的開發流程。在開啟一個新專案或接手一份新需求時，可以先與 AI 進行討論與提問，釐清需求、限制條件、系統架構與驗收標準，接著整理成一份較完整的規格計畫書。當規格文件建立完成後，就能再請 AI 根據規格拆解後續的 tasks、實作步驟以及驗證項目。這種方式比起過去直接叫 AI 修改程式碼更加穩定，因為 AI 在每次實作前都能重新閱讀規格文件，確保後續修改方向與原本需求一致。

我認為這點非常重要，因為如果沒有明確的 spec 或 task 文件，AI 很容易在不同輪對話中產生不一致的想法。有時候雖然最後能生成一個勉強可以執行的專案，但內部架構可能變得混亂，甚至可能違反原本專案的設計方式。相反地，如果一開始就先建立完整的規格與任務拆解，後續每次實作都能讓 AI 依照文件進行小規模修改，再逐步檢查結果是否正確，就能有效

降低 AI 隨意改動架構或偏離需求的風險。這也讓整個開發流程更接近正式的軟體工程流程，而不是單純把程式丟給 AI 後等待結果。

在本次作業中，我也明顯感受到 AI 在偵錯方面的幫助。當程式出現問題時，可以透過截圖、錯誤訊息，或是用文字描述哪個功能不符合規格、哪個操作沒有反饋、畫面顯示結果和預期不同等方式回饋給 AI。AI 便能根據這些資訊重新檢視相關檔案，分析可能的錯誤原因，並提出修改方式。例如 UI 互動問題有時很難只靠程式碼看出來，但如果搭配實際操作後的截圖與行為描述，AI 就能更快理解問題發生的情境，進而協助定位錯誤。